Lecture 5 : Reconnaître le matériau

CNN embarqués et optimisation : quantification, élagage, distillation, NAS, projet EcoTri 4.0

Le modèle fait 3 Mo et la Portenta n'a que 1 Mo de RAM. Que faire ?

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

Apprendre et mesurer le coût | CNN légers | Compresser | Concevoir pour la cible | Arbitrer : Pareto

- **Apprendre et mesurer le coût** : *Pourquoi un CNN, et comment chiffrer ce qu'il coûte à la carte ?*
- **CNN légers** : *Comment diviser le calcul par 8 sans perdre la précision ?*
- **Compresser** : *Peut-on stocker un poids sur 8 bits, en supprimer 80 %, ou apprendre d'un plus grand modèle ?*
- **Concevoir pour la cible** : *Réutiliser ImageNet, chercher l'architecture automatiquement, compiler sans interpréteur.*
- **Arbitrer : Pareto** : *Quel modèle choisir, et comment le défendre dans le rapport J5 ?*

Objectifs

Niveau de Bloom visé : analyser et optimiser

À la fin de cette lecture, vous serez capables de :

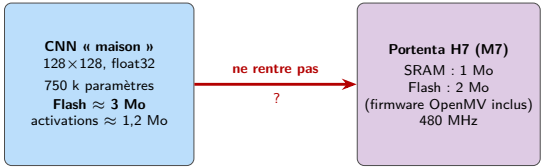
- **Calculer** paramètres, MACs, Flash et pic de RAM d'un CNN, couche par couche.
- **Expliquer** la convolution séparable, le bloc résiduel inversé de MobileNetV2, les hyperparamètres α et résolution, et le principe de FOMO.
- **Appliquer** la quantification INT8 (PTQ, QAT), l'élagage, la distillation et le transfer learning ; **prévoir** leur effet.
- **Situer** la recherche d'architecture (NAS, MCUNet), l'EON Compiler et CMSIS-NN dans la chaîne de déploiement.
- **Lire** un front de Pareto et **argumenter** le choix d'un modèle sous contraintes.

Le fil rouge et le livrable

- La station **voit** (L2), **parle** (L3), **écoute** (L4) ; elle doit maintenant **reconnaître le matériau** : carton, métal, plastique, verre.
- Jalon **J5** : **rapport d'optimisation** comparant trois modèles et choix argumenté, face à la référence à règles J2.

Le problème déclencheur

Le modèle fait 3 Mo et la Portenta n'a que 1 Mo de RAM



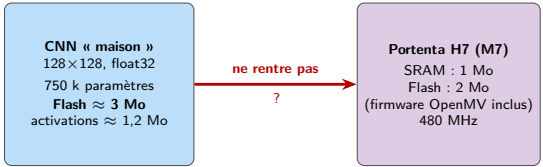
Cahier des charges SP5 : Flash modèle $\leq$ 600 Ko, RAM $\leq$ 250 Ko, latence $\leq$ 100 ms, exactitude $\geq$ référence à règles J2 + 15 points

À vous (5 min, par équipe)

1. Combien d'octets pour 750 k poids en float32 ? En INT8 ?
2. Si la RAM est le problème, est-ce bien les poids qui l'occupent ?
3. Citez trois leviers pour faire « rentrer » le modèle.

Le problème déclencheur

Le modèle fait 3 Mo et la Portenta n'a que 1 Mo de RAM



Cahier des charges SP5 : Flash modèle $\leq$ 600 Ko, RAM $\leq$ 250 Ko, latence $\leq$ 100 ms, exactitude $\geq$ référence à règles J2 + 15 points

À vous (5 min, par équipe)

1. Combien d'octets pour 750 k poids en float32 ? En INT8 ?
2. Si la RAM est le problème, est-ce bien les poids qui l'occupent ?
3. Citez trois leviers pour faire « rentrer » le modèle.

Réponse

3 Mo $\rightarrow$ **750 Ko** en INT8 ($\times 4$). Les poids vont en **Flash** ; la RAM est remplie par les **activations** (cartes intermédiaires). Leviers : architecture plus légère, quantification, élagage, distillation, résolution d'entrée, compilateur.

Apprendre et mesurer le coût

Pourquoi un CNN, et comment chiffrer ce qu'il coûte à la carte ?

- Seuils LAB et blobs : très rapides (quelques ms), interprétables, mais **fragiles** : une bouteille verte en plastique ressemble à du verre, un carton taché à du métal rouillé.
- Chaque nouveau cas impose une nouvelle règle : la complexité croît **plus vite** que la précision.
- Un CNN apprend des caractéristiques **hiérarchiques** (bords → textures → parties) directement depuis les exemples (LeCun et al., 1998 ; Zeiler & Fergus, 2014).

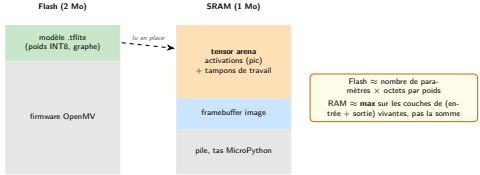
	Règles J2	**CNN**
Précision (4 classes)	55–65 %	85–92 %
Latence	$<$ 5 ms	20–100 ms
Mémoire	$<$ 10 Ko	100–600 Ko
Données	aucune	centaines/classe
Explicabilité	forte	faible

! le CNN doit **justifier** son coût par un gain mesuré sur la **même** base de test que J2.

Pour une convolution $K \times K$, M canaux en entrée, N en sortie, carte de sortie $H \times W$:

$$\#\text{param} = K^2MN + N \qquad \#\text{MACs} = HWK^2MN$$

- **Paramètres** → **Flash** ($\times$ 4 octets en float32, $\times$ 1 en INT8).
- **MACs** (multiplication-accumulation) → **latence** et énergie.
- **Pic de RAM** : max sur les couches de la taille des tenseurs vivants (entrée + sortie).
- **Latence mesurée** sur la cible : la seule qui compte au final.



MACs ≠ latence : accès mémoire, opérateurs non optimisés et cache comptent autant (Banbury et al., MicroNets, 2021).

Couche	Sortie	Paramètres	MACs	Activ. entrée+sortie (INT8)
Conv 3×3, 16, stride 2	48×48×16	448	1,0 M	27,6 + 36,9 = **64,5 Ko**
Conv 3×3, 32, stride 2	24×24×32	4 640	2,7 M	36,9 + 18,4 = 55,3 Ko
Conv 3×3, 64, stride 2	12×12×64	18 496	2,7 M	18,4 + 9,2 = 27,6 Ko
GlobalAvgPool	64	0	–	9,2 Ko
Dense 4	4	260	256	≈ 0
Total		**23,8 k** (24 Ko INT8)	**6,4 M**	**pic 64,5 Ko**

- Les **premières** couches fixent la RAM (grandes cartes) ; les **dernières** concentrent les paramètres.
- À 480 MHz avec CMSIS-NN (≈ 1–2 MAC/cycle en INT8 SIMD) : 6,4 M MACs ⇒ **4 à 15 ms**.

Question flash

?

On remplace le GlobalAvgPool par Flatten ($12 \times 12 \times 64 = 9216$) suivi de Dense 4. Combien de paramètres ajoute-t-on ? Quel risque ?

Question flash

? On remplace le GlobalAvgPool par Flatten ($12 \times 12 \times 64 = 9216$) suivi de Dense 4. Combien de paramètres ajoute-t-on ? Quel risque ?

Réponse

$9216 \times 4 + 4 = \mathbf{36\ 868}$ paramètres : plus que tout le reste du réseau ! Risque de **sur-apprentissage** avec peu de données, et Flash $\times 2{,}5$. Le GlobalAvgPool (Lin et al., *Network in Network*, 2014) est le choix par défaut en TinyML.

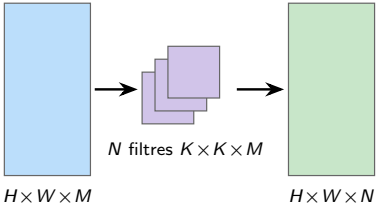
CNN légers

Comment diviser le calcul par 8 sans perdre la précision ?

CNN légers

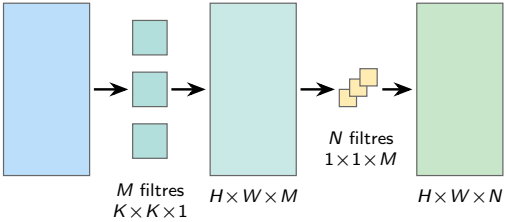
La convolution séparable en profondeur

Convolution standard



MACs = $HW\,K^2MN$

Convolution séparable en profondeur



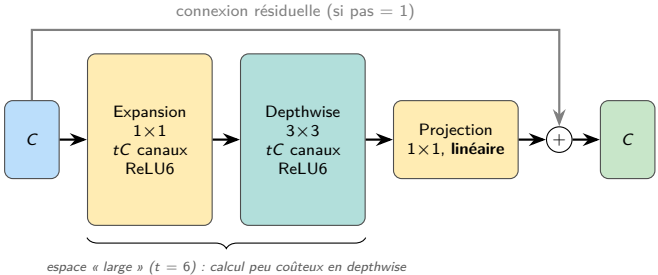
MACs = $HW\,K^2M$ (depthwise) $+HW\,MN$ (pointwise)

Rapport = $\frac{1}{N} + \frac{1}{K^2}$ soit, pour $K = 3$, $N = 64$: $\approx$ **0,13** (8 fois moins de calcul)

Introduite par Sifre (2014), popularisée par Xception (Chollet, CVPR 2017) et MobileNet (Howard et al., Google, 2017).

CNN légers

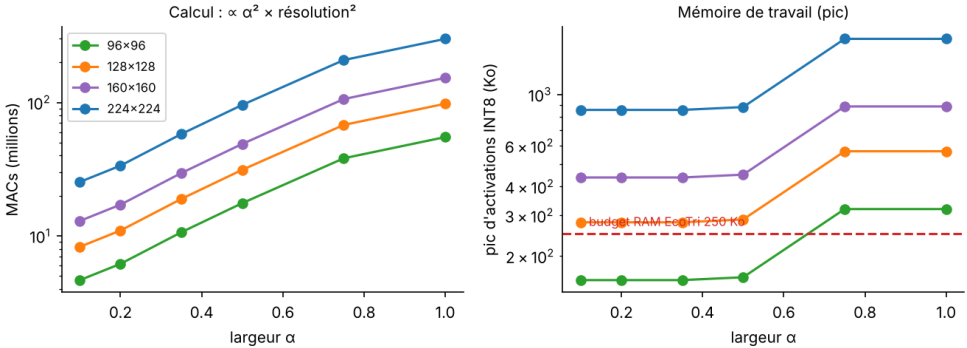
MobileNetV1 puis V2 : le bloc résiduel inversé



- **V1** (2017) : empilement de convolutions séparables ; 4,2 M param., 569 M MACs, 70,6 % ImageNet.
- **V2** (Sandler et al., CVPR 2018) : on **élargit** ($\times 6$), on filtre en depthwise, on **projette** sans activation.
- Projection **linéaire** : un ReLU dans un espace étroit détruit de l'information.
- ReLU6 (bornée à 6) : plage connue $\Rightarrow$ quantification INT8 plus facile.
- V2 1.0 : 3,5 M param., 300 M MACs, 72 % ImageNet.

CNN légers

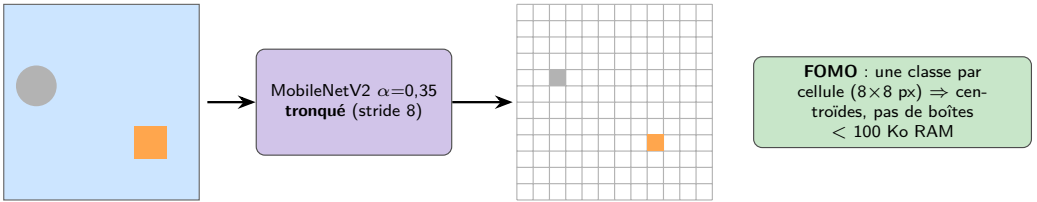
Deux boutons de réglage : la largeur α et la résolution



- α multiplie le nombre de canaux : paramètres et MACs $\propto \alpha^2$; la résolution ρ : MACs et activations $\propto \rho^2$, paramètres inchangés.
- Calculé (script `gen_images.py`) : MobileNetV2 $\alpha = 0{,}35$ en 96×96 : **0,39 M param.**, **10,7 M MACs**, pic $\approx$ **160 Ko** : il rentre ; en 224×224 : 58 M MACs, 860 Ko : il ne rentre plus.

CNN légers

Classification, détection, FOMO



- **Classification** : un objet centré par image → une classe. Cas d'EcoTri si la XIAO déclenche la capture au bon moment.
- **Détection** (SSD, YOLO) : boîtes + classes ; trop lourd pour un M7 (plusieurs Mo).
- **FOMO** (Edge Impulse, 2022) : segmentation grossière par cellule, centroïdes des objets ; 30 fois plus léger qu'un SSD MobileNet, plusieurs objets par image.
- Limite : pas de taille d'objet, objets collés fusionnés.

Question flash

?

Une couche depthwise 3×3 + pointwise de 96 vers 96 canaux, sur une carte 12×12. Comparez ses MACs à ceux d'une convolution 3×3 standard.

Question flash

?

Une couche depthwise 3×3 + pointwise de 96 vers 96 canaux, sur une carte 12×12. Comparez ses MACs à ceux d'une convolution 3×3 standard.

Réponse

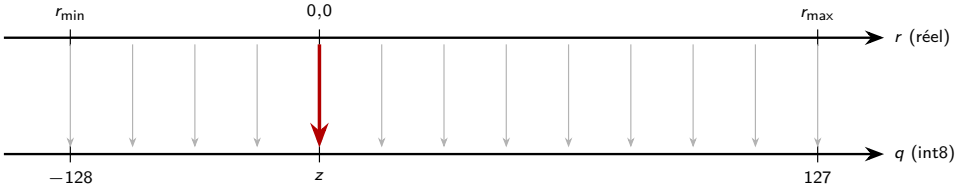
Standard : $144 \times 9 \times 96 \times 96 =$ **11,9 M**. Séparable : $144 \times 9 \times 96 + 144 \times 96 \times 96 = 0{,}12 + 1{,}33 =$ **1,45 M**. Rapport $0{,}12 = 1/96 + 1/9$: la **pointwise** domine (92 % du coût) ; c'est elle qu'optimise CMSIS-NN.

Compresser

Peut-on stocker un poids sur 8 bits, en supprimer 80 %, ou apprendre d'un plus grand modèle ?

Compresser

Quantification : principe affine (uniforme)



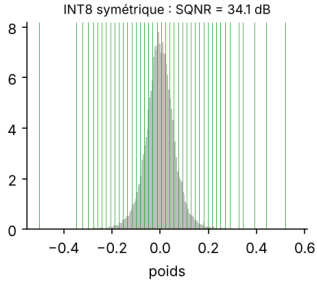
$$q = \text{clamp}\left(\text{round}(r/s) + z, -128, 127\right) \qquad \hat{r} = s(q - z)$$

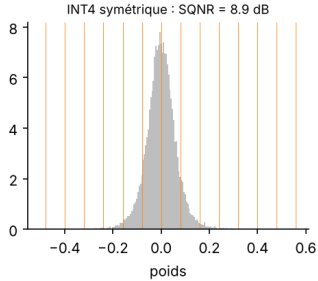
$$s = \frac{r_{max} - r_{min}}{255} \text{ (échelle)}, \quad z = \text{point zéro entier : le réel 0 est } \textbf{exactement} \text{ représentable (padding, ReLU)}$$

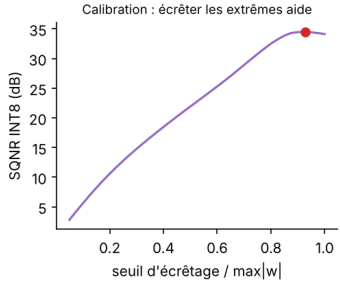
- Jacob et al. (Google, CVPR 2018) : inférence **entièrement entière** ; une multiplication de matrices INT8 accumule en INT32, puis une remise à l'échelle par un entier et un décalage.
- Spécification TFLite : poids INT8 **symétriques par canal** ($z = 0$), activations INT8 **asymétriques par tenseur**.

Compresser

Ce que coûte la quantification : l'erreur d'arrondi





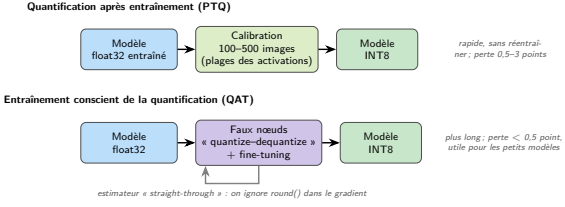


- Erreur d'arrondi uniforme : variance $s^2/12$; chaque bit en moins coûte ≈ 6 dB de SQNR.
- INT8 : bruit négligeable pour la plupart des CNN ; INT4 : perte visible sans QAT.

! les valeurs extrêmes dilatent l'échelle s et écrasent les petites valeurs : on **écrête** (percentile, entropie KL, MSE) au lieu de prendre min/max (Krishnamoorthi, 2018 ; Nagel et al., 2021).

Compresser

PTQ et QAT




```
conv = tf.lite.TFLiteConverter\
       .from_keras_model(model)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
def rep():                  # calibration
    for x in calib.take(200):
        yield [x]
conv.representative_dataset = rep
conv.target_spec.supported_ops = \
  [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type  = tf.int8
conv.inference_output_type = tf.int8
open("m_int8.tflite","wb").write(conv.convert())
```

QAT : `tfmot.quantization.keras.quantize_model(model)`

puis quelques époques.

Compresser

Bénéfices de l'INT8 sur un Cortex-M7

	float32	**INT8**
Taille poids	4 o	1 o ($\times 4$)
Activations	4 o	1 o ($\times 4$)
MAC par instruction	1 (FPU)	2–4 (SIMD `SMLAD`)
Latence typique	1	0,2–0,3
Exactitude	réf.	$-0{,}5$ à -2 pts

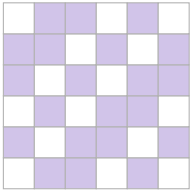
- Les instructions DSP de l'Armv7E-M traitent deux produits 16 bits par cycle ; CMSIS-NN étend les INT8 en INT16 et les exploite (Lai, Suda & Chandra, 2018).
- Énergie : une addition INT8 coûte ≈ 30 fois moins qu'une addition float32 et un accès DRAM 200 fois plus qu'une opération (Horowitz, ISSCC 2014).

> **!** sur MCU, l'INT8 n'est pas une option : c'est le **défaut**.

Compresser

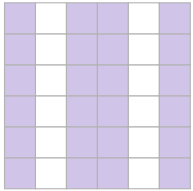
Élagage (pruning)

Non structuré



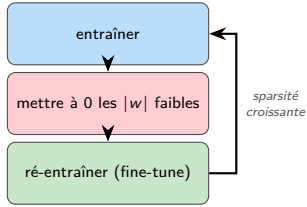
poids nuls épars : matrice creuse, gain réel seulement avec un moteur creux

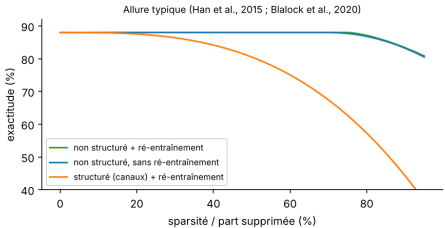
Structuré (filtres / canaux)



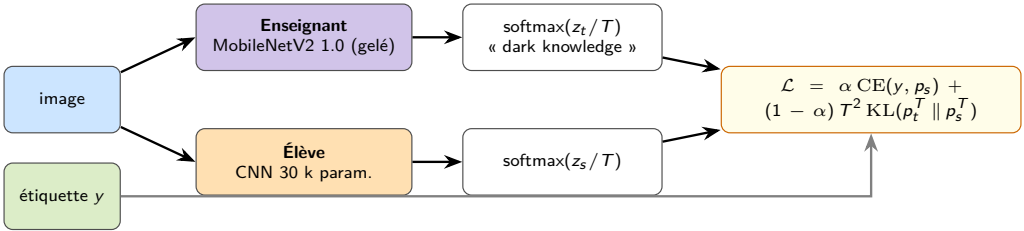
colonnes entières supprimées : couche plus étroite, gain direct en RAM et en MACs

Procédure itérative





- LeCun (*Optimal Brain Damage*, 1990) ; Han et al. (Stanford, *Deep Compression*, ICLR 2016) : élagage + quantification + Huffman, $\times 35$ à $\times 49$.
- Sur MCU, seul l'élagage **structuré** réduit RAM et latence sans moteur creux.



- Hinton, Vinyals & Dean (2015) : la sortie « adoucie » ($T = 2$ à 5) de l'enseignant indique les **ressemblances** entre classes (verre proche de plastique transparent, loin de carton).
- L'élève apprend plus vite et mieux qu'avec les seules étiquettes ; gain typique de 1 à 3 points pour un petit élève.
- Combinable avec la quantification : l'enseignant float32 guide un élève en QAT.

Question flash

?

Votre MobileNetV2 0,35 INT8 fait 380 Ko de poids et 160 Ko de pic de RAM. Vous élaguez 50 % des poids en non structuré, sans moteur creux. Que gagnez-vous en Flash, en RAM, en latence ?

Question flash

? Votre MobileNetV2 0,35 INT8 fait 380 Ko de poids et 160 Ko de pic de RAM. Vous élaguez 50 % des poids en non structuré, sans moteur creux. Que gagnez-vous en Flash, en RAM, en latence ?

Réponse

Rien sur la cible : les zéros sont stockés et multipliés comme les autres poids. Gain seulement après compression du fichier (téléchargement OTA) ou avec un moteur creux. Pour gagner réellement : élagage **structuré** (canaux) ou α plus petit.

Concevoir pour la cible

Réutiliser ImageNet, chercher l'architecture automatiquement, compiler sans interpréteur.

Concevoir pour la cible

Transfer learning : ne pas partir de zéro

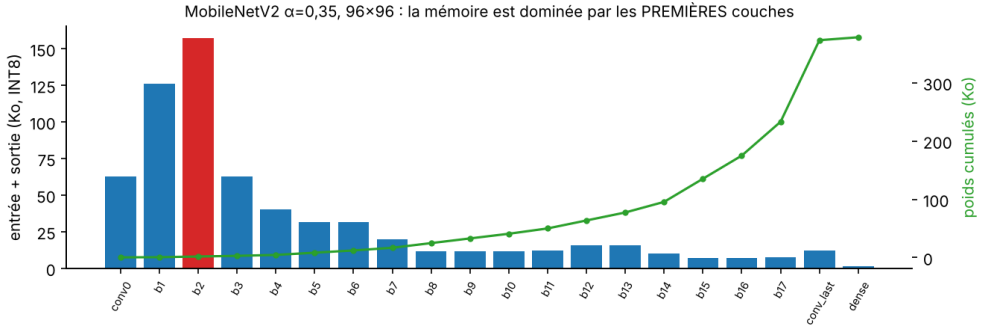
- On reprend un MobileNetV2 pré-entraîné sur ImageNet (1,28 M images) : ses couches basses détectent déjà bords et textures (Yosinski et al., NeurIPS 2014).
- **Étape 1** : geler la base, entraîner la tête (GAP + Dense) ; **étape 2** : dégeler les derniers blocs, taux d'apprentissage ÷ 10.
- Quelques centaines d'images par classe suffisent : TrashNet (Yang & Thung, Stanford CS229, 2016) = 2527 images, 6 classes.

```
base = tf.keras.applications.MobileNetV2(
   input_shape=(96,96,3), alpha=0.35,
   include_top=False, weights="imagenet")
base.trainable = False
x = base(inp, training=False)
x = GlobalAveragePooling2D()(x)
x = Dropout(0.2)(x)
out = Dense(4, activation="softmax")(x)
```

! TrashNet : fond blanc, éclairage studio ; la station : tapis, ombres. On **complète** avec des images de la station (TP5).

Concevoir pour la cible

Le pic de mémoire est au début du réseau



- MCUNetV2 (Lin et al., MIT, NeurIPS 2021) : exécuter les premières couches **par tuiles** (patch-based inference) divise le pic par 4 à 8.
- Plus simple : réduire la résolution d'entrée ou le nombre de canaux de la première couche.

Concevoir pour la cible

Recherche d'architecture neuronale (NAS) et MCUNet

- NAS : un algorithme explore un **espace** d'architectures (profondeurs, largeurs, noyaux) et en optimise un critère (Zoph & Le, 2017 ; MnasNet, Tan et al., 2019).
- **MCUNet** (Lin et al., MIT Han Lab, NeurIPS 2020) : conception **conjointe** du réseau (TinyNAS) et du moteur d'inférence (TinyEngine).
- TinyNAS : (1) choisir l'espace de recherche qui tient dans la mémoire du MCU ; (2) entraîner un super-réseau (*once-for-all*) ; (3) recherche évolutive sous contraintes.

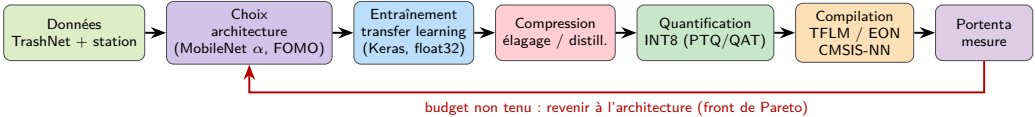
ImageNet sur STM32H743	**Top-1**
MobileNetV2 + TFLM (rentre)	$\approx$ 50 %
MCUNet (TinyNAS + TinyEngine)	70,7 %

Ordres de grandeur publiés (Lin et al., 2020, 2021), 512 Ko SRAM, 2 Mo Flash.

! l'architecture optimale **dépend de la cible** : il n'y a pas de « meilleur modèle » dans l'absolu.

Concevoir pour la cible

Compiler : interpréteur TFLM, EON Compiler, CMSIS-NN



- **TFLite Micro** : interprète le graphe `.tflite` (FlatBuffer) ; souple, mais métadonnées et résolveur d'opérateurs en RAM.
- **EON Compiler** (Edge Impulse) : génère du C++ qui appelle directement les noyaux ; typiquement 25–55 % de RAM et 35 % de Flash en moins.
- **CMSIS-NN** : noyaux INT8 optimisés pour Cortex-M (SIMD, Helium sur M55) ; utilisés **automatiquement** par TFLM et EON.
- **OpenMV** : module `ml` (TFLM intégré au firmware) ; le modèle est chargé dans le tas ou le framebuffer.

Question flash

?

Votre modèle INT8 en 160×160 dépasse la RAM de 120 Ko. Classez trois solutions de la moins à la plus coûteuse en effort et en précision.

Question flash

? Votre modèle INT8 en 160×160 dépasse la RAM de 120 Ko. Classez trois solutions de la moins à la plus coûteuse en effort et en précision.

Réponse

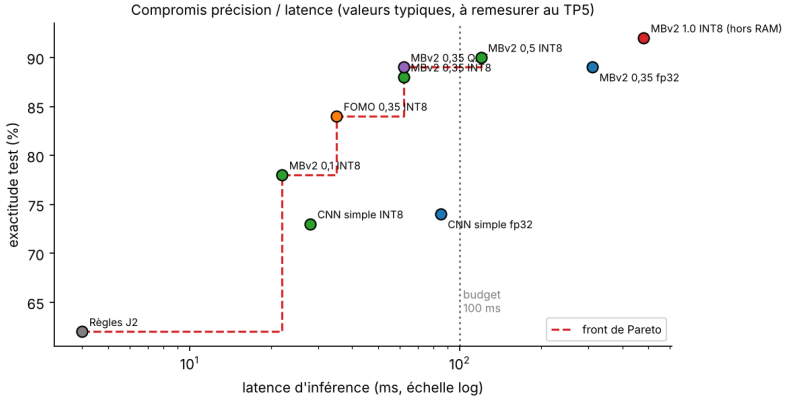
(1) **EON Compiler** au lieu de TFLM : aucun réentraînement, souvent suffisant. (2) Résolution **128×128** : pic $\times(128/160)^2 = 0{,}64$, réentraînement, perte 1–2 points. (3) α plus petit ou NAS : plus de travail et de perte. On mesure après chaque étape.

Arbitrer : Pareto

Quel modèle choisir, et comment le défendre dans le rapport J5 ?

Arbitrer : Pareto

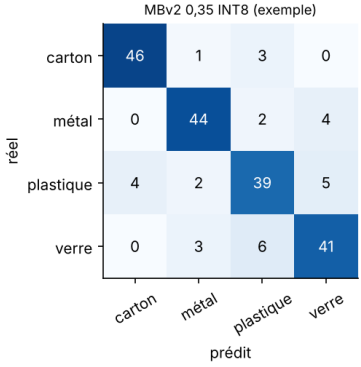
Lire un front de Pareto



- Un modèle est **dominé** s'il existe un autre modèle meilleur sur tous les critères (ici : plus précis **et** plus rapide). On ne choisit que sur le **front**.
- Le choix sur le front dépend des contraintes : budget de 100 ms $\Rightarrow$ MobileNetV2 0,35 INT8 ou 0,5 INT8.

Arbitrer : Pareto

Évaluer honnêtement le classifieur de matériaux

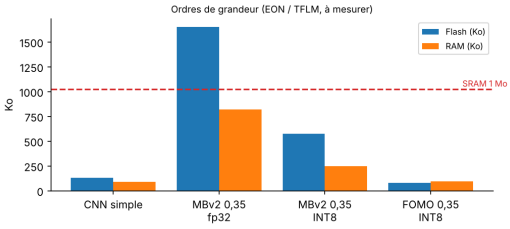


- Les confusions ont un **sens physique** : plastique transparent ↔ verre, métal brillant ↔ verre.
- Rapporter : exactitude équilibrée, rappel par classe, matrice de confusion, sur une base de test **issue de la station** (pas seulement TrashNet).
- Mesurer le modèle INT8 **sur la carte** : la quantification et le prétraitement (RGB565, redimensionnement) peuvent changer les résultats.

! du verre dans le bac plastique abîme la ligne de recyclage : le rappel « verre » compte plus que la moyenne.

Arbitrer : Pareto

Le rapport d'optimisation (jalon J5)



Plan imposé

1. Contraintes (Flash, RAM, latence, précision, rappel verre).
2. Données : TrashNet + station, découpage par session.
3. Trois modèles : CNN simple, MBv2 0,35 float32, MBv2 0,35 INT8.
4. Tableau : param., MACs, Flash, RAM, latence carte, exactitude.
5. Front de Pareto + référence J2.
6. Choix argumenté et limites.

Question flash

?

La Portenta doit aussi faire tourner le modèle audio de la lecture 4 (pic 80 Ko). Il reste 400 Ko de RAM après le firmware et le framebuffer. Proposez une configuration de vision et justifiez-la.

Question flash

?

La Portenta doit aussi faire tourner le modèle audio de la lecture 4 (pic 80 Ko). Il reste 400 Ko de RAM après le firmware et le framebuffer. Proposez une configuration de vision et justifiez-la.

Réponse

Les deux modèles ne tournent pas en même temps : les arenas peuvent être **partagées** (exécution séquentielle), sinon 80 + pic vision $\leq$ 400 Ko. MobileNetV2 0,35, 96×96, INT8 (pic $\approx$ 160 Ko, 10,7 M MACs, $\approx$ 50–70 ms), transfer learning ImageNet $\rightarrow$ TrashNet $\rightarrow$ station, EON ; QAT si la perte INT8 dépasse 1 point ; FOMO si plusieurs objets par image.

À retenir

- **Coût** : paramètres $\rightarrow$ Flash ; MACs $\rightarrow$ latence ; activations $\rightarrow$ pic de RAM (au début du réseau) ; toujours **mesurer** sur la cible.
- **Architecture** : convolution séparable $(1/N + 1/K^2)$, bloc résiduel inversé, α et résolution ; FOMO pour plusieurs objets.
- **Quantification INT8** : $q = \text{round}(r/s) + z$; PTQ avec calibration, QAT si nécessaire ; $\times 4$ en mémoire, $\times 3$–5 en vitesse.
- **Élagage** : utile sur MCU s'il est structuré. **Distillation** : un grand modèle enseigne au petit.
- **Cible** : transfer learning, NAS (MCUNet), EON Compiler et CMSIS-NN.
- **Décision** : front de Pareto, contraintes, coût des erreurs ; comparer à la référence J2.

Glossaire et références

Terme	Définition courte
MAC	multiplication + accumulation
α	multiplicateur de largeur
PTQ	quantif. après entraînement
QAT	entraînement conscient de la quantif.
SQNR	rapport signal/bruit de quantif.
KD	distillation des connaissances
NAS	recherche d'architecture
EON	compilateur Edge Impulse
Pareto	ensemble des non-dominés

- A. Howard et al., « MobileNets », arXiv :1704.04861, 2017 ; M. Sandler et al., « MobileNetV2 », CVPR 2018.
- F. Chollet, « Xception », CVPR 2017.
- B. Jacob et al., « Quantization and Training of NNs for Efficient Integer-Arithmetic-Only Inference », CVPR 2018.
- R. Krishnamoorthi, « Quantizing deep convolutional networks for efficient inference : A whitepaper », arXiv :1806.08342, 2018 ; M. Nagel et al., « A White Paper on Neural Network Quantization », arXiv :2106.08295, 2021.
- S. Han, H. Mao, W. Dally (Stanford), « Deep Compression », ICLR 2016 ; D. Blalock et al. (MIT), « What is the State of Neural Network Pruning ? », MLSys 2020.
- G. Hinton, O. Vinyals, J. Dean, « Distilling the Knowledge in a Neural Network », 2015.
- J. Lin et al. (MIT), « MCUNet », NeurIPS 2020 ; « MCUNetV2 », NeurIPS 2021 ; cours MIT 6.5940 *TinyML and Efficient Deep Learning* (S. Han).
- C. Banbury et al. (Harvard), « MicroNets », MLSys 2021 ; V. Janapa Reddi et al., *Machine Learning Systems*, mlsysbook.ai (Harvard CS249r).
- L. Lai, N. Suda, V. Chandra, « CMSIS-NN », arXiv :1801.06601, 2018.
- G. Thung, M. Yang (Stanford CS229), « Classification of Trash for Recyclability Status », 2016.

Prochaine étape

Tutorial 5 : modèle TFLite INT8 sur la Portenta, mesures de latence et de mémoire. **TP 5** : trois modèles, Pareto, jalon J5.